

Webbasierte Aktien-Tracking-Anwendung



Aufgabenstellung · Lösungskonzept · Arbeitspakete · Zeitplan

Präsentation: 25. März 2026 Bleron Beqiri & Dominik Peschke

Wir geben Impulse



Agenda

01

Aufgabenstellung

Projekthintergrund & Anforderungen

02

Lösungskonzept

Architektur, Use Cases, Design Patterns & UI, Prototyp

03

Arbeitspakete

Strukturierte Aufteilung der Entwicklungsaufgaben

04

Zeitplan

Geplante Zeiteinteilung & Meilensteinplan

Aufgabenstellung

Ausgangssituation

- Der Auftraggeber wünscht eine moderne Webanwendung für Privatanleger zur zentralen Verwaltung von Aktieninvestitionen.
- Im Fokus steht die transparente Nachverfolgung der realen Performance auf Basis separater Einzahlungen – nicht nur der gesamten bzw. Durchschnittswerte.
- Bisher fehlt ein auf einzelne Käufe ausgelegtes, geeignetes Werkzeug, welches den Gewinn/Verlust korrekt berechnet, historisch visualisiert und mit Referenzwerten vergleicht.

Kernprobleme

Keine detaillierte Rendite-Berechnung

Gewinn/Verlust auf Basis von einzelnen Käufen bisher schwer erkennbar

Fehlende Verlaufsdarstellung

Historischer Gewinn/Verlust über bestimmte Zeitintervalle

Keine Inflationskorrektur

Reale Kaufkraftentwicklung nicht einsehbar

Kein Referenzvergleich

Aktienprofit vgl. mit EZB-Zinsentwicklung

Funktionale Anforderungen

Minimalziele



Webanwendung (Frontend + Backend)



Anbindung externer Kurs-API



Gewinn-Tracking via Einzahlungen



Historische Gewinn/Verlust-Anzeige

Wunschziele



Portfolio-Tracking pro Benutzer



Inflationsbereinigte Anzeige



Vergleich mit EZB-Zins

Zusatzfunktionen



Dynamische Gewinnmitnahme- &
Verlustminimierungs Benachrichtigungen

Nicht-funktionale Anforderungen

Performance

- Seitenaufbau < 2 Sekunden
- API-Antwortzeit < 500ms
- Multiuserfähigkeit

Sicherheit

- Passwörter gehasht (BCrypt)
- HTTPS-Kommunikation
- Schutz vor SQL-Injection

Wartbarkeit

- Schichtenarchitektur (MVC)
- DAO/DTO Design-Pattern
- Unit-Tests für Kernlogik

Usability

- Benutzerfreundliche Oberfläche angestrebt
- Navigation orientiert sich an gängigen Web-Konventionen
- Rückmeldungen bei Fehleingaben vorgesehen

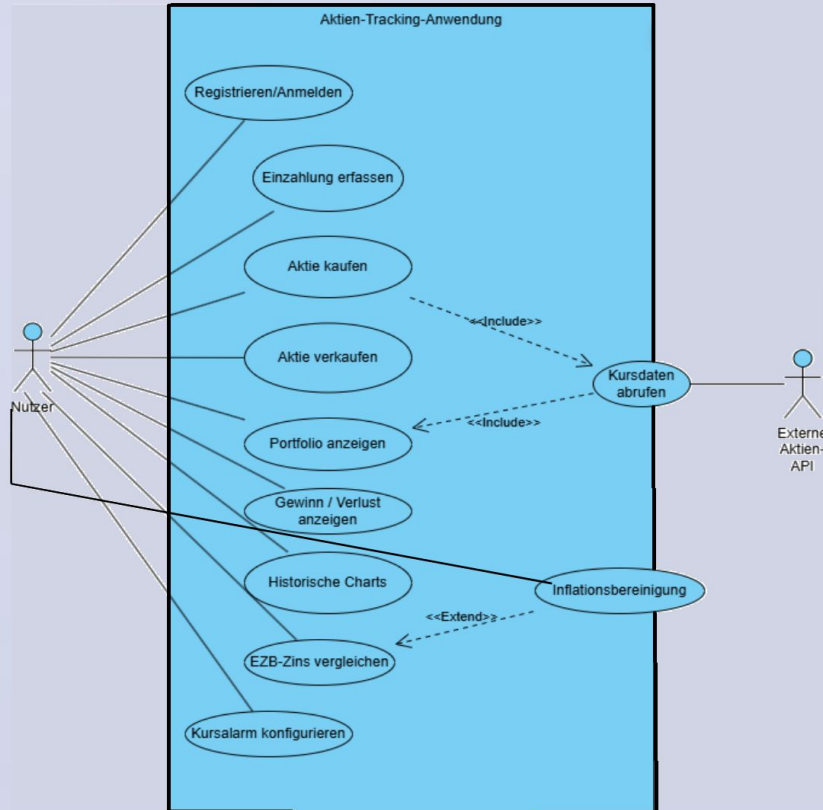
Verfügbarkeit

- Deployment als JAR
- Lauffähig auf Standard-Server
- Datenbankbackup-Konzept

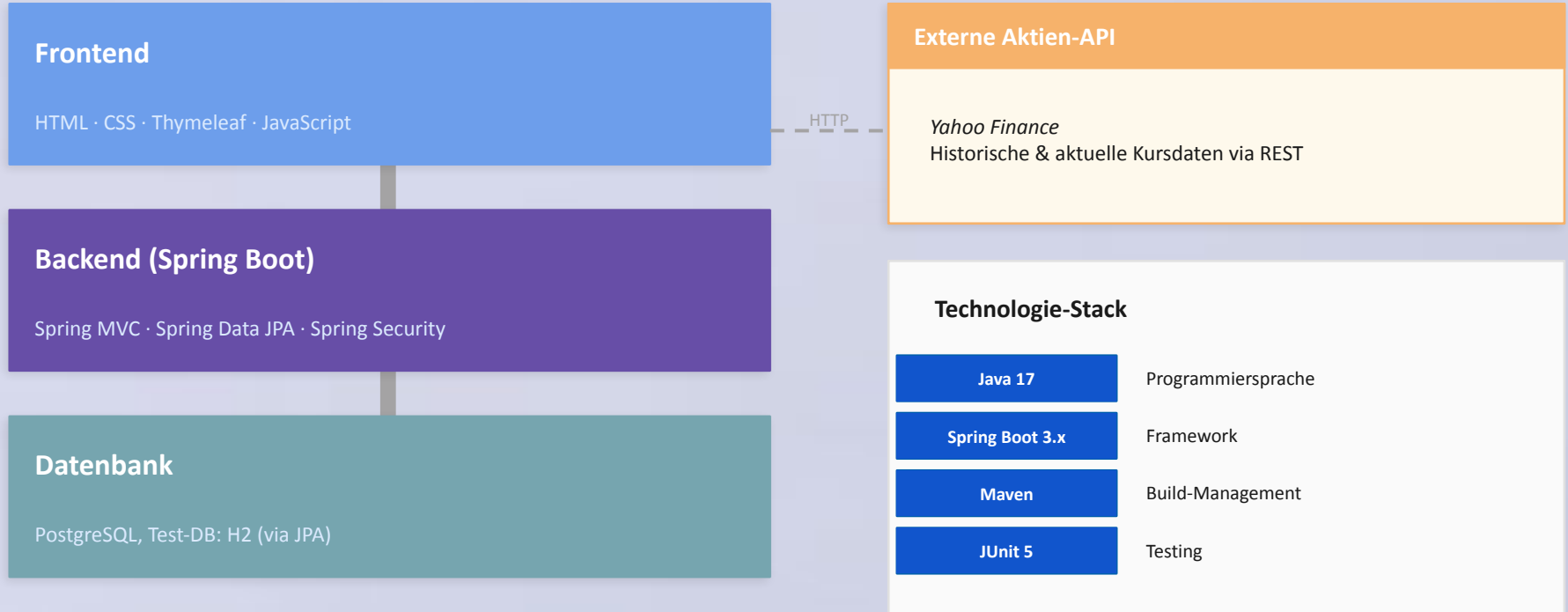
Portabilität

- Plattformunabhängig (Java)
- Maven-Build reproduzierbar
- Konfiguration via .properties

Use-Case-Diagramm



Systemarchitektur



Thymeleaf – Template Engine für Spring Boot

Was ist Thymeleaf?

Server-seitige Template Engine

HTML-Seiten werden auf dem Server generiert und fertig an den Browser gesendet.

Nahtlose Spring Boot Integration

Wird direkt von Spring MVC unterstützt – kein separates Setup nötig.

Natural Templates

HTML bleibt gültig und im Browser direkt einsehbar – kein proprietäres Format.

Daten aus dem Backend

Controller übergibt Model-Objekte, Thymeleaf rendert sie in HTML.

Beispiel: Portfolio-Ansicht

```
<table>
  <thead>
    <tr>
      <th th:text="#{msgs.headers.name}">Name</th>
      <th th:text="#{msgs.headers.price}">Price</th>
    </tr>
  </thead>
  <tbody>
    <tr th:each="prod: ${allProducts}">
      <td th:text="${prod.name}">Oranges</td>
      <td th:text="${#numbers.formatDecimal(prod.price, 1, 2)}">0.99</td>
    </tr>
  </tbody>
</table>
```

Warum Thymeleaf statt z.B. React/Vue?

- Kein separater Frontend-Build nötig
- Direkte Integration mit Spring Security
- Geringere Projektkomplexität
- Ideal für Server-Side-Rendering

Spring Boot – Das Backend-Framework

Was ist Spring Boot?

Opinionated Framework

Vorkonfigurierte Defaults – kein XML-Setup nötig, direkt lauffähig.

Eingebetteter Server

Tomcat läuft direkt in der JAR – kein separater App-Server nötig.

Spring Ecosystem

MVC, Security, Data JPA und mehr – alles integriert.

Dependency Injection

@Autowired verwaltet Abhängigkeiten automatisch.

Beispiel: REST-Controller

```
@RestController
@RequestMapping("/portfolio")
public class PortfolioController {
    @Autowired
    private PortfolioService service;
    @GetMapping("/{id}")
    public PortfolioDTO get(@PathVariable Long id) {
        return service.getPortfolioDTO(id);
    }
}
```

Warum Spring Boot?

- Kein aufwändiges XML-Konfigurationssetup
- Produktionsreif in Minuten
- Weite Community & gute Dokumentation
- Ideal für REST-APIs und Web-Anwendungen

Entwurfsmuster: DAO & DTO

DAO – Data Access Object

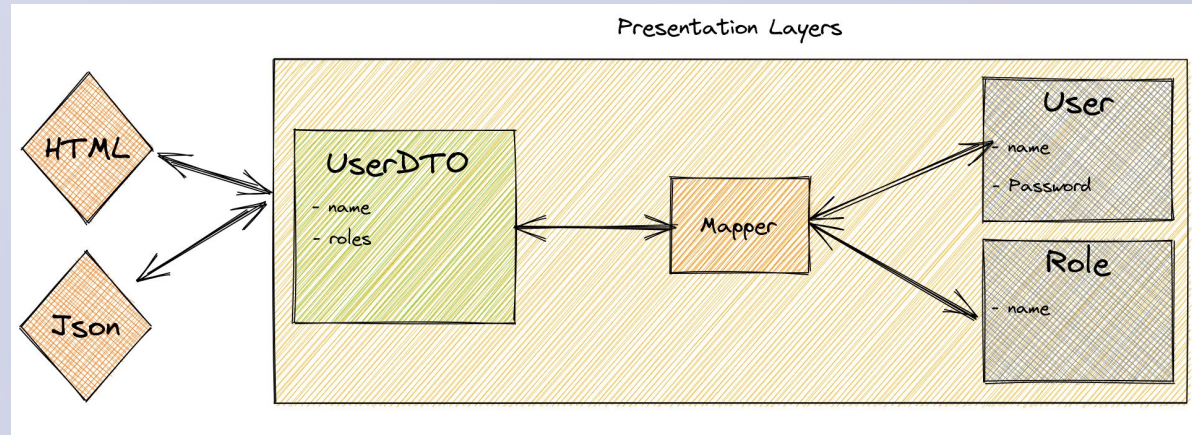
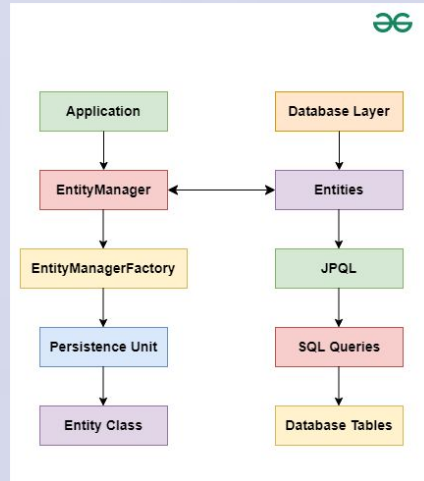
Zweck

Kapselt Datenbanklogik und trennt sie von der Geschäftslogik. Jede Entität erhält ein eigenes DAO-Interface (Spring JPA Repository).

DTO – Data Transfer Object

Zweck

Überträgt nur die benötigten Daten zwischen Schichten (Backend → Frontend). Schützt Entitäten vor direkter Exposition.



Externe Aktien-API – Yahoo Finance & Alpha Vantage

Yahoo Finance API

Weit verbreitete, informell verfügbare REST-API für Aktien-, ETF- und Indexdaten. Kein offizieller Key nötig – Zugriff über inoffizielle Endpunkte oder Bibliotheken (z.B. yahoo-finance2).

Vorteile

- Kostenlos nutzbar
- Große Datentiefe (historisch)
- Bekannte Quelle

Einschränkungen

- Inoffiziell – kann sich jederzeit ändern
- Kein SLA / Support

Alpha Vantage API

Offizieller API-Anbieter mit kostenloser Einstiegsmöglichkeit. Liefert Echtzeit- und historische Kurse, technische Indikatoren und Fundamentaldaten via standardisiertem REST-Interface.

Vorteile

- Offizieller API-Key
- Gute Dokumentation & stabil
- CSV & JSON Support

Einschränkungen

- Kostenlos: 25 Aufrufe/Tag

Integration im Projekt

1

API-Request

Spring @Service ruft REST-Endpoint auf

2

JSON-Parsing

Umformatierung

3

Speicherung

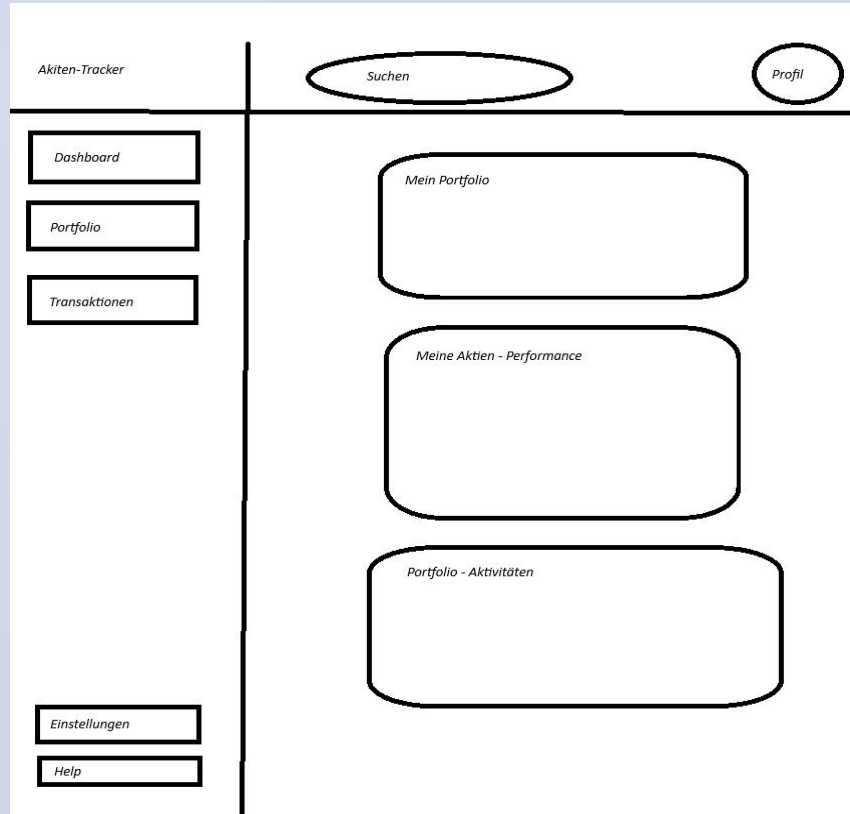
StockPrice wird per DAO in DB persistiert

4

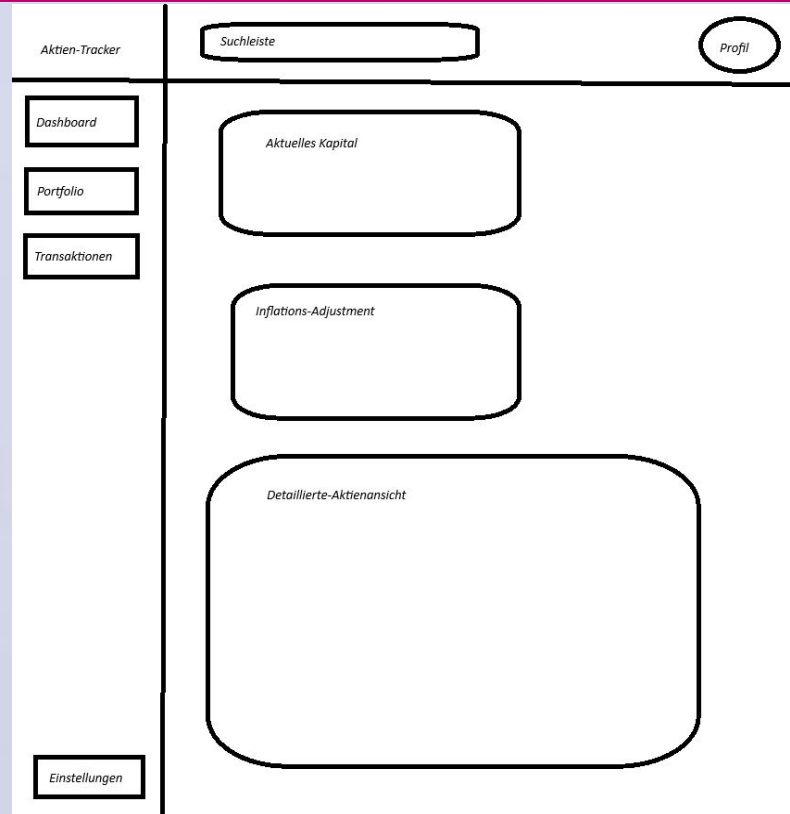
Darstellung

Controller gibt DTO an Thymeleaf-Template weiter

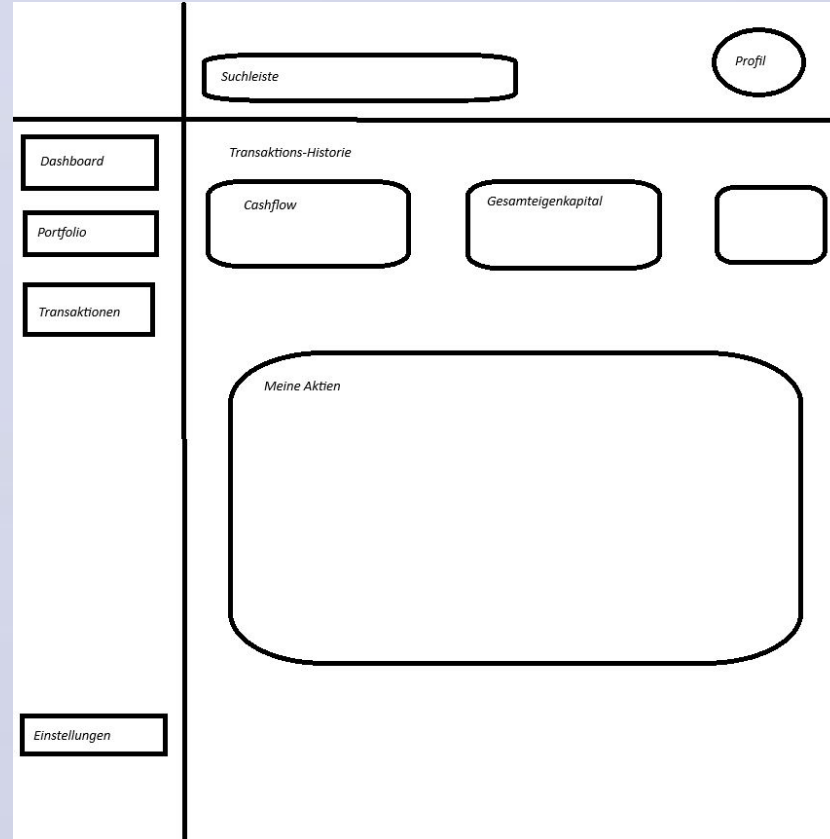
Oberflächenskizzen - Dashboard



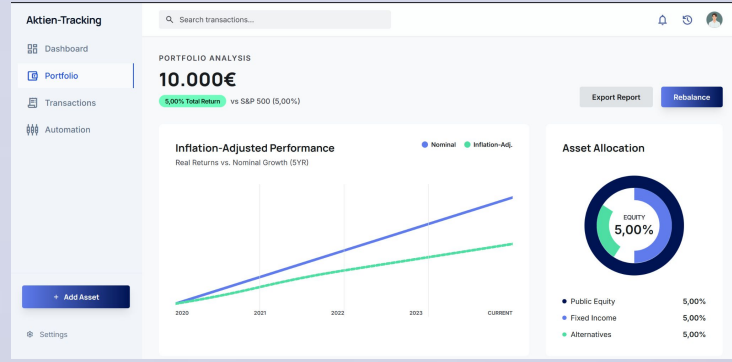
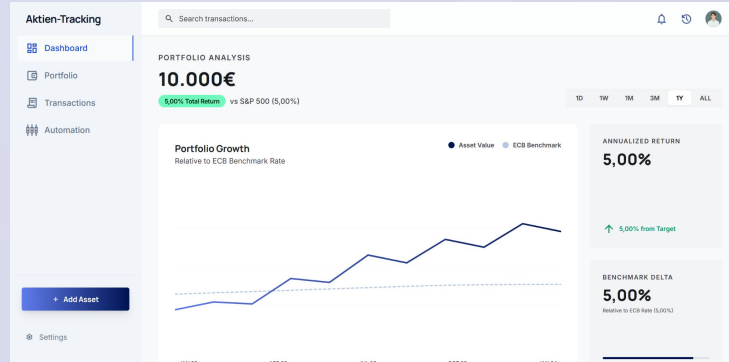
Oberflächenskizzen - Portfolio



Oberflächenskizzen - Transactions



Prototyp Vorstellung



Aktien-Tracking

Search transactions...

Transactions

Historical audit of your equity and liquidity flow.

Export CSV + Add Transaction

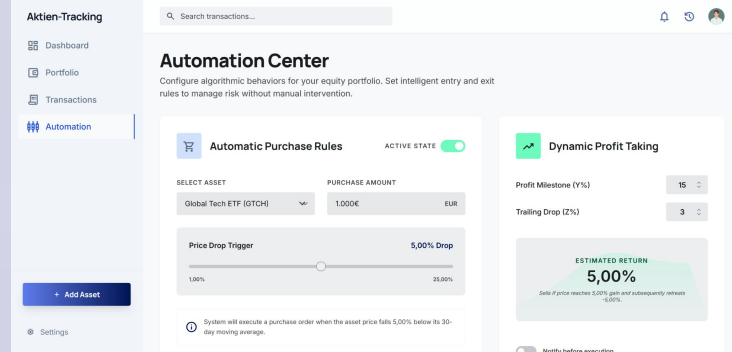
All Transactions Deposits Stock Purchases

▼ Last 30 Days

DATE	ASSET	TYPE	QUANTITY	PRICE	TOTAL AMOUNT	STATUS
Oct 24, 2023 14:33:03T	Apple Inc.	Buy Order	42	1.000€	-10.000€	SETTLED
Oct 22, 2023 09:19:03T	USD Cash Balance	Deposit	—	—	+10.000€	SETTLED
Oct 20, 2023 16:02:03T	NVIDIA Corp	Sell Order	42	1.000€	+10.000€	SETTLED
Oct 19, 2023 11:43:03T	Bitcoin (Spot)	Buy Order	42	1.000€	-10.000€	PENDING
Oct 18, 2023 15:10:03T	Tesla, Inc.	Buy Order	42	1.000€	-10.000€	SETTLED

+ Add Asset

Settings



Arbeitspakete

AP1 – Setup & Infrastruktur

- Spring Boot initialisieren & Projekt aufsetzen
- DB-Schema & JPA-Entities
- Prototypen erstellen & ggfs. anpassen

AP2 – Authentifizierung

- Login / Registrierung
- Spring Security Setup
- Session-Management

AP3 – Frontend – UI

- Dashboard & Portfolio-View
- Charts & Visualisierung
- Responsive Design

AP4 – Backend – Kernlogik

- REST-API (CRUD)
- Gewinn/Verlust-Berechnung
- Externe API-Anbindung

AP5 – Wunsch- & Zusatzfunktionen

- Inflationsbereinigung
- EZB-Zinsvergleich
- Auto-Kauf / Gewinnmitnahme

AP6 – Testing & Deployment

- Unit- & Integrationstests
- Deployment vorbereiten
- Abschlussarbeit

Verbindlicher Zeitplan



Zusammenfassung

Aufgabenstellung

Webanwendung zur transparenten Aktien- und Gewinn-Verfolgung für Privatanleger

Lösungskonzept

Java + Spring Boot, Thymeleaf, externe APIs, PostgreSQL

Arbeitspakete

Setup, Auth, Backend, Frontend, Wunschziele und Testing

Zeitplan

Verbindlicher 6-Wochen-Plan unterteilt in Meilensteine

Vielen Dank für Eure Aufmerksamkeit – Fragen?